

CCES – College of Computer Engineering and Sciences

Prince Sattam Bin Abdulaziz University

CS600: Advanced AI

Searching State Spaces

Chapter 2 — Uninformed, Informed, and Local Search

Dr. Khalil Chebil

Artificial Intelligence Course

Chapter Outline | What we will cover

Search Problems & State Spaces
Uninformed Search
Informed Search

Local Search
Genetic Algorithms

Learning Objectives

- ▶ Represent problems as **state-space graphs**
- ▶ Master **uninformed** search: BFS, DFS, UCS, bidirectional
- ▶ Master **informed** search: greedy best-first and A*
- ▶ Design **admissible** heuristics; apply **local search** and **genetic algorithms**



Section 1

Search Problems & State Spaces

What is Search? | Finding a path from start to goal

Many AI problems reduce to finding a **sequence of actions** transforming an **initial state** into a **goal state**.

Two scenarios

- ▶ **Goal-oriented**: find a path (maze, puzzle)
- ▶ **Optimization**: find a best state (8-queens, scheduling)

Why search matters

- ▶ No closed-form solution
- ▶ Huge solution spaces
- ▶ Needs systematic exploration
- ▶ Trades time, memory, quality

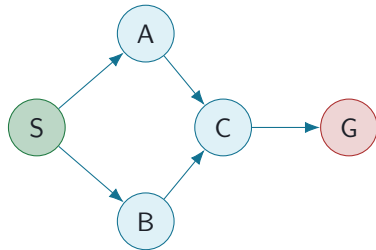
Applications: route planning ■ puzzles ■ game playing ■ scheduling ■ robot navigation

State Space as a Graph | Nodes, edges, paths

- ▶ **Node** — a state
- ▶ **Edge** — an action / transition
- ▶ **Path** — sequence from start to goal

State-space size explodes

8-Puzzle	$9! = 362,880$
8-Queens	$\binom{64}{8} \approx 4.4 \times 10^8$
Rubik's Cube	$\approx 4.3 \times 10^{19}$
Chess	$> 10^{43}$



Large spaces can't be stored — explore states dynamically.

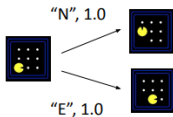
State-Space Graph vs. Search Tree | The tree can be far larger

Aspect	State-Space Graph	Search Tree
Definition	All states & transitions	Paths explored by the algorithm
Duplicates	Each state once	Same state may repeat
Size	Fixed by problem	Can be infinitely larger
Cycles	May contain cycles	Unrolled into branches

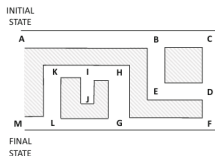
Consequence

A cycle ($a \leftrightarrow b$) makes the tree infinite. Search algorithms must track **visited states** to avoid infinite loops.

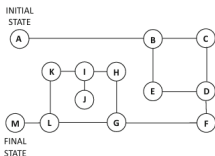
Classic Search Problems | Modeling states and actions



Pacman: position + food; move N/E/S/W

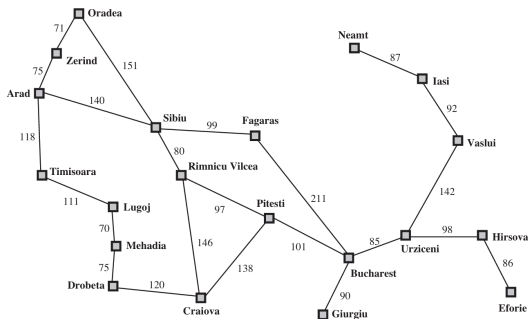


(a) A maze



(b) Graphical state-space abstraction

Maze: position; move in 4 directions



Romania: current city; roads with distances; start Arad
→ goal Bucharest



Section 2

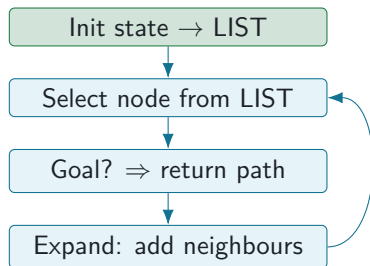
Uninformed Search

Generic Search Framework | One template, many strategies

Components

- ▶ **LIST** — frontier (nodes to explore)
- ▶ **VISIT** — already explored
- ▶ **pred** — predecessors for path recovery
- ▶ **Selection strategy** — defines the algorithm

FIFO $\Rightarrow$ BFS LIFO $\Rightarrow$ DFS priority $\Rightarrow$ UCS / A*



Breadth-First Search (BFS) | Level-by-level, FIFO queue

Explores all nodes at depth d before depth $d+1$.

- ▶ Insert at **end**, remove from **front** (FIFO)
- ▶ Finds the **shallowest** goal

Good for

Shallow goals; optimal when all step costs are equal.

Cost

Memory heavy — stores the whole frontier.

Property	Value
Complete	Yes
Optimal	Yes* (equal costs)
Time	$\mathcal{O}(b^d)$
Space	$\mathcal{O}(b^d)$

b : branching factor d : depth of goal

Depth-First Search (DFS) | Go deep, LIFO stack

Explores as deep as possible, then backtracks.

- ▶ Insert / remove at **end** (LIFO)
- ▶ Expands most recent node first

Good for

Deep solutions, memory-limited settings.

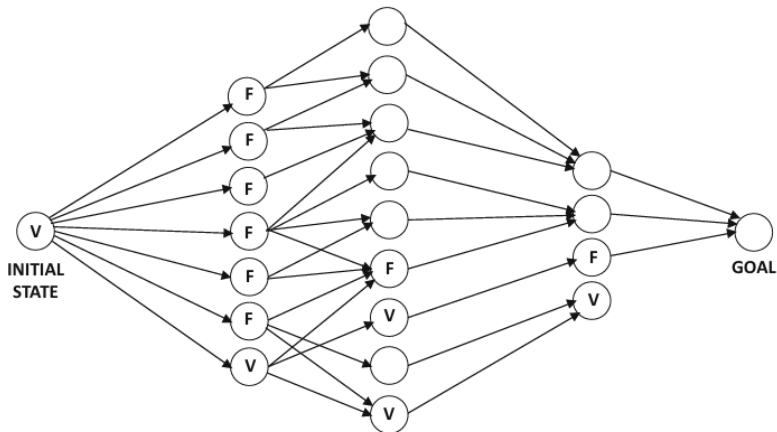
Risk

Can loop forever (needs cycle detection); not optimal.

Property	Value
Complete	No (with cycles)
Optimal	No
Time	$\mathcal{O}(b^m)$
Space	$\mathcal{O}(bm)$

m : maximum depth of the tree

BFS vs. DFS | Frontier shape trade-off



(a) Depth-first search

Uniform-Cost Search (UCS) | BFS for weighted graphs

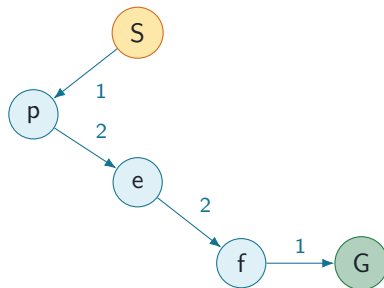
Expand the node with the **lowest path cost** $g(n)$.

- ▶ Priority queue ordered by $g(n)$
- ▶ $g(n)$ = sum of edge costs from start
- ▶ Optimal for any non-negative costs

Complexity

$\mathcal{O}(b^{C^*/\varepsilon})$ time and space, where C^* is the optimal cost and ε the minimum edge cost.

UCS $\equiv$ BFS when all edge costs are equal.



Cheapest-first expansion order

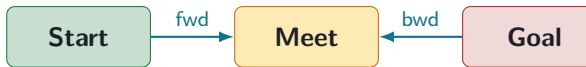
Bidirectional Search | Meet in the middle

Run two searches: **forward** from start and **backward** from goal; stop when they meet.

Speed-up

$$\mathcal{O}(b^d) \rightarrow \mathcal{O}(b^{d/2})$$

$$b=10, d=6 : 10^6 \rightarrow 2 \times 10^3$$



Needs predecessors and a meeting test; goal must be explicit.

Uninformed Search: Summary | Choose by structure and resources

Algorithm	Compl.	Opt.	Time	Space
BFS	Yes	Yes*	$\mathcal{O}(b^d)$	$\mathcal{O}(b^d)$
DFS	No	No	$\mathcal{O}(b^m)$	$\mathcal{O}(bm)$
UCS	Yes	Yes	$\mathcal{O}(b^{C^*/\varepsilon})$	$\mathcal{O}(b^{C^*/\varepsilon})$
Bidirectional	Yes	Yes*	$\mathcal{O}(b^{d/2})$	$\mathcal{O}(b^{d/2})$

Key takeaway: uninformed search is inefficient for large spaces $\Rightarrow$ use **informed search**.

Section 3

Informed Search

A **heuristic** $h(n)$ estimates the cost from n to the nearest goal.

Evaluation functions

- ▶ $g(n)$ — actual cost start $\rightarrow n$
- ▶ $h(n)$ — estimated cost $n \rightarrow$ goal
- ▶ $f(n)$ — estimated total through n

Key properties

- ▶ **Admissible:** $h(n) \leq h^*(n)$
(never overestimates)
- ▶ **Consistent:** $h(n) \leq c(n, n') + h(n')$

Consistency $\Rightarrow$ admissibility (not vice versa).

8-puzzle: misplaced tiles, or sum of Manhattan distances.

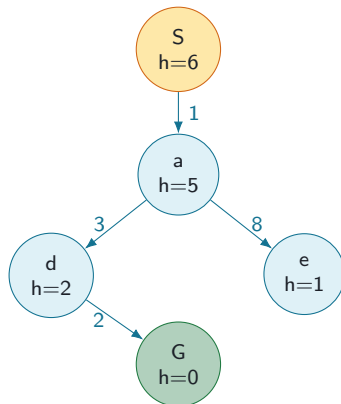
Greedy Best-First Search | $f(n) = h(n)$

Expand the node that *appears* closest to the goal.

- ▶ Priority queue ordered by $h(n)$
- ▶ Ignores the cost already paid, $g(n)$
- ▶ Fast, but **not optimal** / **not complete**

Typical heuristics

straight-line distance ■ Manhattan distance ■ misplaced tiles



Follows smallest h : $S \rightarrow a \rightarrow d \rightarrow G$

A* Search | $f(n) = g(n) + h(n)$

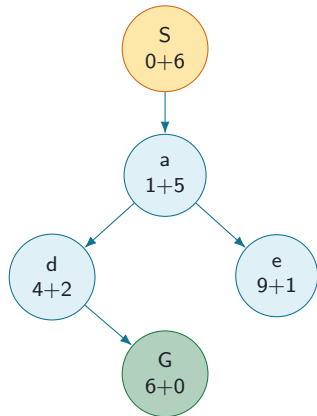
The most widely used informed search:
balances cost **paid** and cost **remaining**.

$$f(n) = \underbrace{g(n)}_{\text{start} \rightarrow n} + \underbrace{h(n)}_{n \rightarrow \text{goal}}$$

- ▶ Priority queue ordered by $f(n)$
- ▶ Expands the most promising *total-cost* node

Optimally efficient

With a given admissible h , no optimal algorithm expands fewer nodes.



Optimal path $S \rightarrow a \rightarrow d \rightarrow G$, $f = 6$

Why A* is Optimal | Admissibility guarantees optimality

Proof sketch

Suppose A* returned suboptimal G_2 (cost C_2), while optimal G has cost $C^* < C_2$. Some node n on the optimal path is unexpanded. Since h is admissible:

$$f(n) = g(n) + h(n) \leq g(n) + h^*(n) = C^* < C_2 = f(G_2).$$

A* expands by increasing f , so n precedes G_2 — contradiction. ■

Terminate correctly

Stop when the goal is **selected for expansion**, not when first generated — otherwise the path may be suboptimal.

Condition

Optimal *iff* h is **admissible** (never overestimates the true cost).

8-Puzzle heuristics

- ▶ h_1 = number of **misplaced tiles**
- ▶ $h_2 = \sum_{\text{tiles}} |\Delta x| + |\Delta y|$
(**Manhattan distance**)

Both admissible; h_2 **dominates** h_1 .

Techniques

Relaxed problems ▪ pattern databases ▪ learning

Heuristic	Avg nodes
None (UCS)	~170,000
Misplaced h_1	~500
Manhattan h_2	~50

Good h : admissible, consistent, accurate, fast.



Section 4

Local Search

Explore the **neighborhood** of the current state; keep no path.

- ▶ State-centric loss functions
- ▶ Incremental refinement
- ▶ Low memory

Ideal for

8-queens, TSP, scheduling, VLSI layout, vehicle routing — the *result* matters, not the route to it.

Hill Climbing | Greedy local improvement

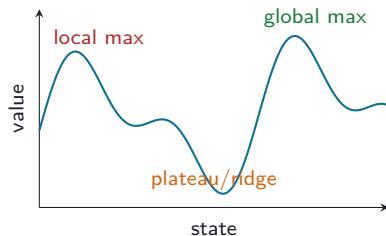
Repeatedly move to the best neighbor until none improves.

- ▶ 8-queens: move a queen to cut attacking pairs
- ▶ TSP: swap two cities to shorten the tour

Gets stuck at

local maxima ■ plateaus ■ ridges

Fixes: random restart, stochastic / first-choice moves.



Simulated Annealing | Accept some bad moves to escape optima

Inspired by metallurgical annealing: allow occasional **uphill** moves.

$$P(\text{accept}) = \begin{cases} 1 & \Delta L < 0 \\ e^{-\Delta L/T} & \Delta L \geq 0 \end{cases}$$

- ▶ High T : explore widely (accept bad moves often)
- ▶ Low T : exploit; $T \rightarrow 0$ becomes hill climbing
- ▶ Keep the **best** solution seen

Cooling schedules

- ▶ Linear: $T_0 - \alpha t$
- ▶ Exponential: $T_0 \gamma^t$
- ▶ Logarithmic: $T_0 / \log t$

✓ escapes local optima ✗ needs schedule tuning

Enhance hill climbing with a **tabu list** of recently visited moves/states that are temporarily forbidden.

- ▶ **Tabu list**: fixed-size recent-move memory
- ▶ **Aspiration**: allow a tabu move if it beats the best-ever
- ▶ **Intensify** promising regions; **diversify** to new ones

Strengths

Escapes local optima deterministically; strong for TSP, scheduling, routing.

Needs tuning of the tabu-list size.

Local Search: Comparison

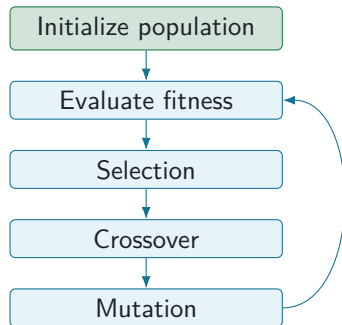
Method	Escapes optima?	Memory	Best for
Hill Climbing	No	None	Simple, quick
Random Restart	Yes	None	Many local optima
Simulated Annealing	Yes (prob.)	None	Large spaces
Tabu Search	Yes (determ.)	Tabu list	Avoiding cycles
Genetic Algorithm	Yes	Population	Complex optimization

Section 5

Genetic Algorithms

Evolve a **population** of solutions over generations.

- ▶ **Chromosome**: encoded solution
- ▶ **Fitness**: solution quality
- ▶ **Selection**: fitter parents reproduce
- ▶ **Crossover**: recombine parents
- ▶ **Mutation**: random diversity



TSP example: chromosome = city permutation; fitness = $-\text{tour length}$; order crossover; swap mutation.

Section 6

Summary

Chapter Summary | Key takeaways

Uninformed

BFS (optimal, memory-heavy), DFS (light, not optimal), UCS (cost-optimal), bidirectional ($b^{d/2}$).

Informed

Greedy (h) is fast but not optimal; A^* ($g+h$) is optimal with an admissible heuristic.

Local & evolutionary

Hill climbing, simulated annealing, tabu search, and genetic algorithms optimize the *final state*.

Looking ahead

Ch. 3: adversarial search and games (minimax, α - β).

Thank you!

Questions & Discussion

Next: Chapter 3 — Multiagent Search